



# Tarea Unidad 4: Programación de Scripts de Administración y Automatización (Curso 25/26)

## 1. DESCRIPCIÓN GENERAL. OBJETIVOS

Esta tarea consolida las competencias de programación dentro del servidor de bases de datos (procedimientos, funciones y disparadores) y su interacción con los metadatos del SGBD para labores de administración.

Para la realización de esta entrega, utilizaremos la base de datos de la "**Liga de Baloncesto**" (liga), cuya estructura de tablas (equipos, jugadores, partidos, estadísticas) debió ser desplegada previamente.

## 2. Bloque I: Estructuras Básicas, Funciones y Control de Flujo (Repaso)

### Ejercicio 1: Procedimiento Sumador

- **Enunciado:** Diseña un procedimiento almacenado `sp_sumar_valores` al cual le pases dos valores numéricos enteros como parámetros de entrada (IN) y devuelva la suma de ambos en una variable de salida (OUT).

### Ejercicio 2: Función Sumadora

- **Enunciado:** Crea una función almacenada `fn_sumar_valores` que tenga exactamente la misma funcionalidad del ejercicio anterior (recibir dos enteros y retornar su suma).

### Ejercicio 3: Control de Signo y Paridad

- **Enunciado:** Crea un procedimiento `sp_analizar_suma` que invoque internamente al procedimiento o función de los ejercicios anteriores. Debe recibir dos números de entrada y devolver dos parámetros de salida:
  - El primer parámetro de salida (OUT) devolverá un carácter representativo: '>' si la suma es positiva, '<' si es negativa, o '=' si es igual a cero.
  - El segundo parámetro de salida (OUT) será un valor booleano (TINYINT(1) o BOOLEAN) que indique si la suma es par (TRUE/1) o impar (FALSE/0).



### Ejercicio 4: Contador de Homónimos

- **Enunciado:** Escribe una función `fn_contar_jugadores_nombre` que reciba como parámetro el nombre de un jugador y devuelva el número de jugadores que se llaman exactamente así dentro de la tabla `jugadores`.

### Ejercicio 5: Bucle de Capitanes

- **Enunciado:** Crea un procedimiento `sp_contar_por_capitan` que emplee una estructura de bucle repetitivo (`WHILE`, `LOOP` o `REPEAT`) para calcular y mostrar por pantalla cuántos jugadores tienen asignado como capitán de equipo a los IDs del 1 al 14 de forma secuencial.

## 3. Bloque II: Estructuras Avanzadas - Cursores

### Ejercicio 6: Cursor de Resultados Límites (Puntuación de 90)

- **Enunciado:** Diseña un procedimiento `sp_partidos_puntuacion_limite` que utilice un **CURSOR** para recorrer los partidos. El procedimiento debe imprimir o visualizar el nombre de los equipos enfrentados y sus respectivos resultados únicamente cuando alguno de los dos equipos haya alcanzado exactamente los 90 puntos en el encuentro.

### Ejercicio 7: Búsqueda del Máximo Salario sin funciones de agregado

- **Enunciado:** Crea un procedimiento `sp_jugador_max_salario` que utilice un **CURSOR** para obtener los nombres de los jugadores y sus salarios. El procedimiento debe calcular y mostrar el nombre del jugador con el salario más alto del club **sin utilizar la función predefinida MAX()**.

### Ejercicio 8: Equipo con Mayor Actividad

- **Enunciado:** Crea un procedimiento `sp_equipo_mas_partidos` que utilice un **CURSOR** para analizar el número de partidos jugados por cada equipo. Debe mostrar el nombre del equipo que más encuentros ha disputado a lo largo de la liga **sin utilizar la función MAX()**.

### Ejercicio 9: Procedimiento con Cursor para Modificación de Salarios (Mantenimiento)

- **Enunciado:** Crea un procedimiento `sp_incrementar_salario_jugadores` con un cursor al que se le pasará un porcentaje o una cantidad fija. El cursor deberá recorrer a todos los jugadores de la base de datos e ir incrementando su salario de forma individual.

## 4. Bloque III: Automatización mediante Disparadores



## (Triggers)

### Ejercicio 10: Control de Salarios Negativos

- **Enunciado:** Crea un disparador `tg_salario_positivo_before_update` que controle la actualización de salarios sobre la tabla jugadores. En caso de que se intente asignar un salario menor o igual a 0, el disparador debe cancelar el cambio manteniendo el salario anterior del jugador de manera silenciosa, o lanzar un error personalizado.

### Ejercicio 11: Actualización Dinámica de Clasificación

- **Enunciado:** Crea un disparador `tg_actualizar_puntos Equipos` de tipo AFTER INSERT sobre la tabla partidos. Cada vez que se inserte un partido, el disparador debe identificar cuál ha sido el equipo vencedor y sumarle de forma automática 1 punto en su casillero de clasificación en la tabla equipos. Realiza pruebas de inserción para verificar su funcionamiento.

### Ejercicio 12: Auditoría Integral de la Tabla Equipos

- **Enunciado:** Diseña un sistema de auditoría para monitorizar los cambios en la tabla equipos.
  1. Crea una tabla de logs llamada `auditoria_equipos` con los campos: `id_log` (PK, Auto), `usuario`, `operacion` (INSERT / UPDATE / DELETE), `fecha_hora` e `id_equipo_afectado`.
  2. Crea tres disparadores (AFTER INSERT, AFTER UPDATE y AFTER DELETE) que alimenten de forma automática esta tabla registrando quién realizó la operación, la fecha exacta y la tabla afectada.

### Ejercicio 13: Restricción de Inserciones por Privilegio de Usuario

- **Enunciado:** Crea un disparador `tg_insert_solo_root` para la tabla equipos. El disparador debe validar si el usuario que está realizando la operación es el administrador supremo (`root@localhost`). Si cualquier otro usuario intenta realizar un insert, el trigger debe abortar la operación utilizando `SIGNAL SQLSTATE '45000'` con un mensaje descriptivo de denegación de privilegios.

## 5. Bloque IV: Administración e Interacción con el Catálogo

### Ejercicio 14: Replicador del Comando Describe (DESC)

- **Enunciado:** Diseña un procedimiento almacenado llamado `sp_describir_tabla` que reciba como parámetro de entrada el nombre de una tabla. El procedimiento debe consultar las



vistas del sistema en `information_schema.columns` para mostrar por pantalla: el nombre de la columna, el tipo de dato, si permite nulos (YES/NO) y su valor predeterminado (reproduciendo de forma personalizada la salida de la orden DESCRIBE o DESC).

## Ejercicio 15: Purga Programada de Partidos Antiguos

- **Enunciado:** Desarrolla un procedimiento `sp_purgar_partidos` al cual se le pasará una fecha como parámetro de entrada. El procedimiento eliminará de forma segura todos los registros de partidos disputados con anterioridad a la fecha indicada para reducir el peso en tablas históricas.

## 6. Rúbrica de Evaluación y Calificación Detallada

La evaluación de la tarea se basará en una puntuación máxima de **10 puntos**, distribuidos de la siguiente manera:

### Distribución de Puntos por Ejercicio:

- **Bloque I: Estructuras Básicas (2.0 puntos en total)**
  - **Ejercicio 1 (0.4 pts):** Definición de parámetros IN/OUT y asignación correcta de la suma.
  - **Ejercicio 2 (0.4 pts):** Estructura correcta de función, cláusula RETURNS y RETURN.
  - **Ejercicio 3 (0.4 pts):** Invocación interna de subprogramas y estructuras condicionales (IF-ELSE).
  - **Ejercicio 4 (0.4 pts):** Consulta condicional sobre tablas dentro de una función con retorno escalar.
  - **Ejercicio 5 (0.4 pts):** Lógica y control del bucle repetitivo (WHILE/LOOP) y salida formateada.
- **Bloque II: Uso de Cursores (3.0 puntos en total)**
  - **Ejercicio 6 (0.75 pts):** Declaración, apertura, lectura recursiva y cierre del cursor con filtrado de datos.
  - **Ejercicio 7 (0.75 pts):** Algoritmo de cálculo de máximo relativo mediante variables de estado sin MAX().
  - **Ejercicio 8 (0.75 pts):** Correcta construcción de la consulta de base del cursor (uniones/agrupaciones) y cálculo del máximo.
  - **Ejercicio 9 (0.75 pts):** Integración de actualización DML de registros fila a fila mediante el cursor.
- **Bloque III: Diseño de Disparadores (3.0 puntos en total)**
  - **Ejercicio 10 (0.75 pts):** Control temporal (BEFORE UPDATE) y manejo seguro del estado de las variables NEW y OLD.
  - **Ejercicio 11 (0.75 pts):** Lógica condicional compleja tras una inserción (AFTER INSERT)



- para identificar al ganador del encuentro.
- **Ejercicio 12 (0.75 pts):** Definición de tabla de auditoría de logs y correcta separación de eventos en tres disparadores distintos.
- **Ejercicio 13 (0.75 pts):** Implementación de la directiva de seguridad controlando la variable del sistema USER() y bloqueando con SIGNAL.
- **Bloque IV: Administración de Metadatos (2.0 puntos en total)**
  - **Ejercicio 14 (1.0 pt):** Consulta dinámica sobre information\_schema.columns y formateo adecuado de la salida.
  - **Ejercicio 15 (1.0 pt):** Purga de registros y control de integridad de datos referenciales en tablas enlazadas.

## 7. Solucionario Oficial de la Tarea

A continuación se detalla la resolución técnica completa de todos los ejercicios planteados para la base de datos liga.

-- Asegurar que trabajamos en el esquema correcto

CREATE DATABASE IF NOT EXISTS liga;

USE liga;

--

=====

===

-- SOLUCIONARIO BLOQUE I: ESTRUCTURAS BÁSICAS

--

=====

===

-- Ejercicio 1: Procedimiento Sumador

DELIMITER //

CREATE PROCEDURE sp\_sumar\_valores(

IN p\_num1 INT,

IN p\_num2 INT,

OUT p\_resultado INT

)

BEGIN

SET p\_resultado = p\_num1 + p\_num2;

END //

DELIMITER ;



-- Ejercicio 2: Función Sumadora

DELIMITER //

CREATE FUNCTION fn\_sumar\_valores(

    p\_num1 INT,

    p\_num2 INT

)

RETURNS INT

DETERMINISTIC

BEGIN

    RETURN p\_num1 + p\_num2;

END //

DELIMITER ;

-- Ejercicio 3: Control de Signo y Paridad

DELIMITER //

CREATE PROCEDURE sp\_analizar\_suma(

    IN p\_val1 INT,

    IN p\_val2 INT,

    OUT p\_signo CHAR(1),

    OUT p\_es\_par BOOLEAN

)

BEGIN

    DECLARE v\_suma INT;

    -- Invocamos la función creada en el Ejercicio 2

    SET v\_suma = fn\_sumar\_valores(p\_val1, p\_val2);

    -- Evaluar signo de la suma

    IF v\_suma > 0 THEN

        SET p\_signo = '>';

    ELSEIF v\_suma < 0 THEN

        SET p\_signo = '<';

    ELSE

        SET p\_signo = '=';

    END IF;

    -- Evaluar paridad utilizando operador módulo (%)

    IF v\_suma % 2 = 0 THEN

        SET p\_es\_par = TRUE;



```
ELSE
    SET p_es_par = FALSE;
END IF;
END //
DELIMITER ;

-- Ejercicio 4: Contador de Homónimos
DELIMITER //
CREATE FUNCTION fn_contar_jugadores_nombre(
    p_nombre VARCHAR(100)
)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_cantidad INT;

    SELECT COUNT(*) INTO v_cantidad
    FROM jugadores
    WHERE nombre = p_nombre;

    RETURN v_cantidad;
END //
DELIMITER ;

-- Ejercicio 5: Bucle de Capitanes
DELIMITER //
CREATE PROCEDURE sp_contar_por_capitan()
BEGIN
    DECLARE v_iterador INT DEFAULT 1;
    DECLARE v_total_jugadores INT;

    -- Bucle para recorrer de forma secuencial del ID 1 al 14
    WHILE v_iterador <= 14 DO
        SELECT COUNT(*) INTO v_total_jugadores
        FROM jugadores
        WHERE id_capitan = v_iterador;

        -- Mostrar por pantalla el resultado de la iteración actual
        SELECT CONCAT('El capitán con ID ', v_iterador, ' lidera a ', v_total_jugadores, ' jugadores.')
```



AS 'Resultado';

```
-- Incrementar el contador del bucle
SET v_iterador = v_iterador + 1;
END WHILE;
END //
DELIMITER ;
```

```
--
=====
===
-- SOLUCIONARIO BLOQUE II: CURSORES
--
=====
===
```

-- Ejercicio 6: Cursor de Resultados Límites (Puntuación de 90)

DELIMITER //

CREATE PROCEDURE sp\_partidos\_puntuacion\_limite()

BEGIN

-- Variables del cursor

DECLARE v\_eq\_local VARCHAR(100);

DECLARE v\_eq\_visitante VARCHAR(100);

DECLARE v\_puntos\_local INT;

DECLARE v\_puntos\_visitante INT;

DECLARE fin\_bucle INT DEFAULT FALSE;

-- Declaración del cursor sobre partidos

DECLARE cursor\_partidos CURSOR FOR

SELECT elocal, evisitante, puntos\_local, puntos\_visitante  
FROM partidos;

-- Manejador de fin de datos

DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin\_bucle = TRUE;

OPEN cursor\_partidos;

bucle\_partidos: LOOP



```
    FETCH cursor_partidos INTO v_eq_local, v_eq_visitante, v_puntos_local,
v_puntos_visitante;

    IF fin_bucle THEN
        LEAVE bucle_partidos;
    END IF;

    -- Verificar si alguno de los dos alcanzó exactamente 90 puntos
    IF v_puntos_local = 90 OR v_puntos_visitante = 90 THEN
        SELECT CONCAT(v_eq_local, '(', v_puntos_local, ') vs (', v_puntos_visitante, ') ',
v_eq_visitante) AS 'Partido Límite 90';
    END IF;
END LOOP bucle_partidos;

CLOSE cursor_partidos;
END //
DELIMITER ;

-- Ejercicio 7: Búsqueda del Máximo Salario sin funciones de agregado
DELIMITER //
CREATE PROCEDURE sp_jugador_max_salario()
BEGIN
    DECLARE v_nombre VARCHAR(100);
    DECLARE v_salario DECIMAL(10,2);
    DECLARE v_max_nombre VARCHAR(100) DEFAULT "";
    DECLARE v_max_salario DECIMAL(10,2) DEFAULT 0.00;
    DECLARE fin_bucle INT DEFAULT FALSE;

    DECLARE cursor_salarios CURSOR FOR
        SELECT nombre, salario FROM jugadores;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_bucle = TRUE;

    OPEN cursor_salarios;

    bucle_salarios: LOOP
        FETCH cursor_salarios INTO v_nombre, v_salario;

        IF fin_bucle THEN
```



```
        LEAVE bucle_salarios;
    END IF;

    -- Algoritmo de comparación clásica
    IF v_salario > v_max_salario THEN
        SET v_max_salario = v_salario;
        SET v_max_nombre = v_nombre;
    END IF;
END LOOP bucle_salarios;

CLOSE cursor_salarios;

-- Retornar el resultado calculado
SELECT v_max_nombre AS 'Jugador Mejor Pagado', v_max_salario AS 'Salario';
END //
DELIMITER ;

-- Ejercicio 8: Equipo con Mayor Actividad
DELIMITER //
CREATE PROCEDURE sp_equipo_mas_partidos()
BEGIN
    DECLARE v_equipo VARCHAR(100);
    DECLARE v_partidos INT;
    DECLARE v_max_equipo VARCHAR(100) DEFAULT "";
    DECLARE v_max_partidos INT DEFAULT -1;
    DECLARE fin_bucle INT DEFAULT FALSE;

    -- Consulta base que cuenta los partidos de cada equipo (tanto local como visitante)
    DECLARE cursor_actividad CURSOR FOR
        SELECT eq.nombre, COUNT(part.id_partido)
        FROM equipos eq
        LEFT JOIN partidos part ON eq.nombre = part.elocal OR eq.nombre = part.evisitante
        GROUP BY eq.nombre;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_bucle = TRUE;

    OPEN cursor_actividad;

    bucle_actividad: LOOP
```



```
FETCH cursor_actividad INTO v_equipo, v_partidos;

IF fin_bucle THEN
    LEAVE bucle_actividad;
END IF;

IF v_partidos > v_max_partidos THEN
    SET v_max_partidos = v_partidos;
    SET v_max_equipo = v_equipo;
END IF;
END LOOP bucle_actividad;

CLOSE cursor_actividad;

SELECT v_max_equipo AS 'Equipo Más Activo', v_max_partidos AS 'Partidos Jugados';
END //
DELIMITER ;

-- Ejercicio 9: Procedimiento con Cursor para Modificación de Salarios
DELIMITER //
CREATE PROCEDURE sp_incrementar_salario_jugadores(
    IN p_incremento_porcentaje DECIMAL(5,2)
)
BEGIN
    DECLARE v_id_jugador INT;
    DECLARE v_salario_actual DECIMAL(10,2);
    DECLARE fin_bucle INT DEFAULT FALSE;

    DECLARE cursor_modificacion CURSOR FOR
        SELECT id_jugador, salario FROM jugadores;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_bucle = TRUE;

    OPEN cursor_modificacion;

    bucle_modificacion: LOOP
        FETCH cursor_modificacion INTO v_id_jugador, v_salario_actual;

        IF fin_bucle THEN
```



```
        LEAVE bucle_modificacion;
    END IF;

    -- Ejecutar la actualización individual del registro actual
    UPDATE jugadores
    SET salario = v_salario_actual * (1 + (p_incremento_porcentaje / 100))
    WHERE id_jugador = v_id_jugador;

END LOOP bucle_modificacion;

CLOSE cursor_modificacion;
END //
DELIMITER ;

--
=====
===
-- SOLUCIONARIO BLOQUE III: DISPARADORES (TRIGGERS)
--
=====
===

-- Ejercicio 10: Control de Salarios Negativos
DELIMITER //
CREATE TRIGGER tg_salario_positivo_before_update
BEFORE UPDATE ON jugadores
FOR EACH ROW
BEGIN
    -- Si el nuevo salario es negativo o nulo, mantener el anterior de manera silenciosa
    IF NEW.salario <= 0 THEN
        SET NEW.salario = OLD.salario;
    END IF;
END //
DELIMITER ;

-- Ejercicio 11: Actualización Dinámica de Clasificación
DELIMITER //
CREATE TRIGGER tg_actualizar_puntos Equipos
```



```
AFTER INSERT ON partidos
FOR EACH ROW
BEGIN
    -- El trigger determina el ganador del partido y suma 1 punto en su casillero
    IF NEW.puntos_local > NEW.puntos_visitante THEN
        UPDATE equipos SET puntos = puntos + 1 WHERE nombre = NEW.elocal;
    ELSEIF NEW.puntos_visitante > NEW.puntos_local THEN
        UPDATE equipos SET puntos = puntos + 1 WHERE nombre = NEW.evisitante;
    END IF;
END //
DELIMITER ;
```

-- Ejercicio 12: Auditoría Integral de la Tabla Equipos

-- 12.1. Creación de la tabla de auditoría

```
CREATE TABLE IF NOT EXISTS auditoria_equipos (
    id_log INT AUTO_INCREMENT PRIMARY KEY,
    usuario VARCHAR(100),
    operacion VARCHAR(20),
    fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    id_equipo_afectado INT
);
```

-- 12.2. Triggers de auditoría individualizados por evento

DELIMITER //

```
CREATE TRIGGER tg_auditoria_equipos_insert
AFTER INSERT ON equipos
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_equipos (usuario, operacion, id_equipo_afectado)
    VALUES (USER(), 'INSERT', NEW.id_equipo);
END //
```

```
CREATE TRIGGER tg_auditoria_equipos_update
AFTER UPDATE ON equipos
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_equipos (usuario, operacion, id_equipo_afectado)
    VALUES (USER(), 'UPDATE', NEW.id_equipo);
END //
```



```
CREATE TRIGGER tg_auditoria_equipos_delete
AFTER DELETE ON equipos
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_equipos (usuario, operacion, id_equipo_afectado)
    VALUES (USER(), 'DELETE', OLD.id_equipo);
END //
DELIMITER ;
```

-- Ejercicio 13: Restricción de Inserciones por Privilegio de Usuario

```
DELIMITER //
CREATE TRIGGER tg_insert_solo_root
BEFORE INSERT ON equipos
FOR EACH ROW
BEGIN
    -- Validar si el usuario actual es root de manera local
    IF USER() != 'root@localhost' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR DE SEGURIDAD: Únicamente el usuario ROOT está
autorizado para añadir equipos.';
    END IF;
END //
DELIMITER ;
```

--

=====

===

-- SOLUCIONARIO BLOQUE IV: ADMINISTRACIÓN Y CATÁLOGO

--

=====

===

-- Ejercicio 14: Replicador del Comando Describe

```
DELIMITER //
CREATE PROCEDURE sp_describir_tabla(
    IN p_nombre_tabla VARCHAR(64)
)
```



BEGIN

-- Selección estructurada de metadatos de la tabla especificada en el esquema activo

SELECT

column\_name AS 'Field',

column\_type AS 'Type',

is\_nullable AS 'Null',

column\_key AS 'Key',

column\_default AS 'Default'

FROM information\_schema.columns

WHERE table\_schema = DATABASE() -- Utiliza el esquema actual activo

AND table\_name = p\_nombre\_tabla;

END //

DELIMITER ;

-- Ejercicio 15: Purga Programada de Partidos Antiguos

DELIMITER //

CREATE PROCEDURE sp\_purgar\_partidos(

IN p\_fecha\_corte DATE

)

BEGIN

-- Eliminación segura de los registros de partidos anteriores a la fecha de corte

DELETE FROM partidos

WHERE fecha < p\_fecha\_corte;

END //

DELIMITER ;